The Journal of Open Source Software

GeoRacoon: A Python package for scalable, parallelized geospatial raster analysis

Simon Landauer [1], **Pascal A. Niklaus** [1], **and Jonas I. Liechti** [2]

1 Department of Evolutionary Biology and Environmental Studies, University of Zurich, 8057 Zurich, Switzerland **2** T4D GmbH, 8045 Zurich, Switzerland

DOI: 10.21105/joss.10797

Software

- Review
- Repository
- Archive

Editor: William Gearty

Reviewers:

- @goergen95
- @nkarasiak

Submitted: 29 May 2026
Published: 16 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

The Python package `GeoRacoon` is aimed at supporting work and analyses with large spatial raster data. It consists of 3 sub-packages: `riogrande`, a class based extension of Rasterio (Gillies & others, 2013) with extended functionalities for tag based raster object management; `convster`, a fully parallelized module for convolution of spatial raster data (e.g., satellite imagery); `coonfit`, a multiple linear regression module using fully parallelized analytical solutions to enable large data analysis.

Statement of need

Analyses of large geospatial raster datasets such as remote sensing-derived imagery (e.g., land-cover maps, vegetation indices) and climate variables frequently involve spatial filtering (convolution) and pixel-wise statistical modelling within their analysis pipelines. When applied to imagery with billions of pixels, these operations become computationally prohibitive without parallelization and out-of-core processing strategies, in particular when performed on local computation infrastructure.

At the time of development, to the authors' best knowledge, no Python package provided a turnkey, parallelized workflow for Gaussian convolution of large categorical rasters with correct handling of *nodata* boundaries. Existing tools either required users to implement parallelization themselves or did not support border-preserving convolution for categorical data. Similarly, fitting pixel-wise multiple linear regression (MLR) across full rasters — where spatial bands serve as predictors and response — lacked an accessible, parallelized Python solution using the analytical normal-equations approach. Finally, working with multi-band rasters carrying rich metadata (e.g., land-cover category labels, temporal identifiers) required manual bookkeeping of band indices; no library offered tag-based band selection out of the box.

`GeoRacoon` addresses these gaps jointly in a common framework. Its three sub-packages evolved from the practical needs of a research project on landscape diversity effects on vegetation productivity (Landauer et al., 2025), where large-scale spatial convolution, derived heterogeneity metrics, and raster-based regression were essential. During development, `riogrande` emerged as an extension of Rasterio (Gillies & others, 2013) to manage large volumes of imagery in an object-oriented, tag-based manner, improving easy usability of remote sensing imagery for broader applications beyond the original research context. The other two sub-packages, `convster` and `coonfit` build upon `riogrande` to implement convolution methods and multiple linear regression in a fully parallelized approach.

The package can be directly installed via `pip install git+https://github.com/GeoRacoon/georacoon`. with a comprehensive documentation including examples of usage available at https://georacoon.readthedocs.io/en/latest/.

State of the field

Several well-established tools address different parts of the geospatial raster processing pipeline. First, Rasterio (Gillies & others, 2013) provides the baseline Python interface for geospatial raster input/output (I/O), built on top of GDAL (GDAL/OGR contributors, 2025), but offers no high-level analytical operations such as convolution or regression. Next, `xarray` (Hoyer & Hamman, 2017) and its geospatial extension `rioxarray` enable lazy, chunked computation via Dask, facilitating the basic handling of large multidimensional arrays. However, they lack analytical capabilityies for spatial filtering or pixel-wise MLR. While `scipy.ndimage` (Virtanen et al., 2020) and `scikit-image` (Walt et al., 2014) provide spatial filters, including Gaussian, both operate in-memory and do not handle geospatial metadata or nodata boundaries, also lacking block-parallel decomposition. Cloud-based platforms, such as Google Earth Engine (Gorelick et al., 2017) can also be accessible via Python API but are tied to proprietary infrastructure and do not support full custom filter implementations and often even limit the incorporation of personal datasets that are not part of the provided catalogue.

`GeoRacoon` stands out by combining block-parallel processing with correct nodata handling, tag-based metadata management, and an object-oriented approach to raster data.

Software design

`GeoRacoon` is organized into three sub-packages with clear separation of concerns: `riogrande` provides the foundation and parallelization infrastructure, `convster` implements spatial convolution and derived metrics, and `coonfit` provides parallelized multiple linear regression. The packages build on Rasterio (Gillies & others, 2013), NumPy (Harris et al., 2020), `scikit-image` (Walt et al., 2014), and SciPy (Virtanen et al., 2020).

Parallelization model.
All computationally intensive operations follow a common block-parallel architecture. Rasters are decomposed into spatial blocks via a view system, with configurable overlap (borders) to prevent edge artifacts in filter operations. Workers are dispatched via Python's `multiprocessing.Pool`, each processing an array block independently, while a dedicated aggregator process collects results from a shared queue and writes them to disk. This producer-consumer pattern implements a flexible parallelization mechanism: by decreasing the block size (lowering the memory requirement of a single concurrent job) and reducing the pool size (reducing the multiplicity of concurrent jobs) the total memory requirement can be reduced drastically, pushing towards a more sequential processing of rasters. In combination with an aggregation mechanism that processes block-by-block, `GeoRacoon` allows a trade-off between RAM and execution time, enabling large-scale raster analysis even on modest hardware.

`riogrande` — tag-based raster management.
The `Source` and `Band` classes extend Rasterio with a structured metadata layer. Band-level metadata is stored as JSON-serialized key-value pairs in a dedicated GeoTIFF tag namespace, enabling tag-based band selection (e.g., querying by land-cover category or date) instead of manual index tracking. Additional functionality includes configurable mask strategies per band, compatibility checks across rasters (CRS, resolution, units), data type conversion with range rescaling, and file compression utilities.

`convster` — parallelized spatial convolution.
The convolution module follows a filter-agnostic design: any callable matching the expected signature can be used as the spatial filter. The default Gaussian filter wraps `skimage.filters.gaussian`, and a custom border-preserving variant (`bpgaussian`) correctly handles *nodata* boundaries through a normalization-correction approach. Kernel dimensions are estimated adaptively from the filter's impulse response, and border sizes are computed automatically to ensure artifact-free block-parallel processing. Beyond raw convolution,

`convster` provides derived spatial metrics including Shannon entropy and an interaction index inspired by the Simpson index, computed from filtered categorical layers.

`coonfit` — parallelized multiple linear regression.
The regression module exploits the decomposability of the normal equations: since $X^TX = \sum_b X_b^T X_b$ and $X^Ty = \sum_b X_b^T y_b$ over spatial blocks b, both terms can be computed in parallel. The workflow thereby proceeds in three stages: (1) parallel accumulation of the block-wise X^TX matrices, (2) serial inversion of the resulting (small) matrix, and (3) parallel computation of the regression coefficients. Rank-deficiency detection, predictor consistency validation, and model evaluation metrics (RMSE, R^2) are provided as part of the pipeline.

Research impact statement

The `GeoRacoon` package was central to carrying out a research project on the effects of landscape diversity on vegetation productivity at a global scale (Landauer et al., 2025). The analyses required Gaussian convolution and heterogeneity metric computation of land-cover data, as well as per-pixel MLR with numerous predictors in billion-pixel multi-band remote sensing imagery. Given the non-availability of cloud based solutions for the datasets as well as the methods, this research would have not been computationally feasible without the provided parallelization in this package.

Beyond this initial application, the package is highly usable in any research involving large geospatial rasters. While `riogrande` can generally assist with the simplified management of multi-band remote sensing raster objects, the parallelization enables large-scale geospatial analysis for researchers who lack access to cloud platforms. Specific use cases range from multi-raster earth observation studies applying spatial filtering to remote sensing imagery, to climate-impact analyses requiring efficient pixel-wise regression. By providing these workflows without any dependency on external computing infrastructure, `GeoRacoon` lowers the barrier to reproducible research.

AI usage disclosure

No generative AI tools were used in the development of this software. AI-assisted tools were used to format some of the source code docstrings and to support the drafting of portions of this manuscript, which were subsequently reviewed and edited by the authors.

Acknowledgements

The initial stages of development for this package were financially supported by the European Union's Horizon 2020 research and innovation programme under the Marie Sklodowska-Curie (MSC) grant agreement No 847585 as well as a grant of the University of Zurich Research Priority Program Global Change and Biodiversity (URPP GCB).

References

GDAL/OGR contributors. (2025). *GDAL/OGR geospatial data abstraction software library*. Open Source Geospatial Foundation. https://doi.org/10.5281/zenodo.5884351

Gillies, S., & others. (2013). *Rasterio: Geospatial raster i/o for Python programmers*. Mapbox. https://github.com/rasterio/rasterio

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, *202*, 18–27. https://doi.org/10.1016/j.rse.2017.06.031

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Pirus, M., Hoyer, S., Krevelen, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hoyer, S., & Hamman, J. (2017). Xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, *5*(1), 10. https://doi.org/10.5334/jors.148

Landauer, S., Altermatt, F., Flury, R., Furrer, R., Isbell, F., Liechti, J. I., Spehn, E., & Niklaus, P. A. (2025). Positive effects of landscape diversity on vegetation productivity at the kilometer scale. *bioRxiv*. https://doi.org/10.1101/2025.09.25.678498

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Walt, S. van der, Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & the scikit-image contributors. (2014). Scikit-image: Image processing in Python. *PeerJ*, *2*, e453. https://doi.org/10.7717/peerj.453